

TUGAS AKHIR PRAKTIKUM

Tugas 1: Sistem Manajemen Proses Buat program C yang mensimulasikan sistem manajemen proses sederhana dengan fitur: Membuat multiple child processes

Setiap process melakukan tugas berbeda (sorting, searching, calculation) Parent process mengkoordinasi dan mengumpulkan hasil Tampilkan PID, execution time, dan status setiap proses

Lahkah Kerja:

1. Buat File proc_manager.c

```
// proc_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <string.h>
#include <time.h>
#include <errno.h>
#define BUF_SIZE 512

// helper: elapsed seconds between two timespecs
double elapsed_sec(struct timespec start, struct timespec end) {
    double s = (double)(end.tv_sec - start.tv_sec);
    double ns = (double)(end.tv_nsec - start.tv_nsec);
    return s + ns * 1e-9;
}
```

// A simple bubble sort (so it's deterministic CPU work)

```
void bubble_sort(int *a, size_t n) {  
    for (size_t i = 0; i < n; ++i)  
        for (size_t j = 0; j + 1 < n - i; ++j)  
            if (a[j] > a[j+1]) {  
                int t = a[j]; a[j] = a[j+1]; a[j+1] = t;  
            }  
}
```

// A simple linear search: returns index or -1

```
int linear_search(int *a, size_t n, int target) {  
    for (size_t i = 0; i < n; ++i)  
        if (a[i] == target) return (int)i;  
    return -1;  
}
```

// Calculation example: sum of primes up to limit (sieve)

```
long sum_primes_up_to(int limit) {  
    if (limit < 2) return 0;  
    char *isprime = calloc(limit+1, 1);  
    if (!isprime) return -1;  
    for (int i = 2; i <= limit; ++i) isprime[i] = 1;  
    long sum = 0;  
    for (int p = 2; p <= limit; ++p) {  
        if (!isprime[p]) continue;  
        sum += p;  
        if ((long)p * p <= limit)  
            for (int q = p*p; q <= limit; q += p) isprime[q] = 0;  
    }  
}
```

```
    free(isprime);  
    return sum;  
}
```

```
int main(void) {  
    // we'll create 3 children: 0=sort, 1=search, 2=calc  
    const int NCH = 3;  
    int pipes[NCH][2];  
    pid_t pids[NCH];  
  
    for (int i = 0; i < NCH; ++i) {  
        if (pipe(pipes[i]) == -1) {  
            perror("pipe");  
            exit(EXIT_FAILURE);  
        }  
    }  
}
```

```
for (int i = 0; i < NCH; ++i) {  
    pid_t pid = fork();  
    if (pid < 0) {  
        perror("fork");  
        exit(EXIT_FAILURE);  
    } else if (pid == 0) {  
        // CHILD  
        // close read ends for this child  
        for (int j = 0; j < NCH; ++j) {  
            if (j == i) continue;  
            close(pipes[j][0]);  
            close(pipes[j][1]);  
        }  
    }  
}
```

```

}

close(pipes[i][0]); // close read end; child writes to pipes[i][1]


struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);


char outbuf[BUF_SIZE];
memset(outbuf, 0, sizeof(outbuf));


if (i == 0) {
    // Sorting task

    int n = 2000;

    int *arr = malloc(sizeof(int) * n);
    if (!arr) { snprintf(outbuf, sizeof(outbuf), "ERR malloc");
    write(pipes[i][1], outbuf, strlen(outbuf)); close(pipes[i][1]); exit(1); }

    // fill descending to make sort take time
    for (int k = 0; k < n; ++k) arr[k] = n - k;

    bubble_sort(arr, n);

    // prepare result: first 5 elements
    char sample[128] = {0};
    int take = 5;
    char tmp[32];
    strcpy(sample, "");
    for (int t = 0; t < take; ++t) {
        snprintf(tmp, sizeof(tmp), "%d%s", arr[t], (t+1<take)?", ":"");
        strncat(sample, tmp, sizeof(sample)-strlen(sample)-1);
    }

    free(arr);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double et = elapsed_sec(start, end);

```

```

        snprintf(outbuf, sizeof(outbuf),
"TASK=sort;PID=%d;TIME=%.6f;RESULT=first5:[%s]\n", getpid(), et,
sample);

        write(pipes[i][1], outbuf, strlen(outbuf));

        close(pipes[i][1]);

        exit(0);
    } else if (i == 1) {

        // Searching task

        int n = 500000; // larger array but linear search will take time

        int *arr = malloc(sizeof(int) * n);

        if (!arr) { snprintf(outbuf, sizeof(outbuf), "ERR malloc"); write(pipes[i][1],
outbuf, strlen(outbuf)); close(pipes[i][1]); exit(1); }

        // fill array with numbers 0..n-1
        for (int k = 0; k < n; ++k) arr[k] = k;

        int target = n - 1; // worst-case (last element)

        int idx = linear_search(arr, n, target);

        free(arr);

        clock_gettime(CLOCK_MONOTONIC, &end);

        double et = elapsed_sec(start, end);

        snprintf(outbuf, sizeof(outbuf),
"TASK=search;PID=%d;TIME=%.6f;RESULT=index=%d\n", getpid(),
et, idx);

        write(pipes[i][1], outbuf, strlen(outbuf));

        close(pipes[i][1]);

        exit(0);
    } else {

        // Calculation task: sum of primes

        int limit = 200000; // moderate heavy CPU

        long sum = sum_primes_up_to(limit);

        clock_gettime(CLOCK_MONOTONIC, &end);

        double et = elapsed_sec(start, end);

```

```

        snprintf(outbuf, sizeof(outbuf),
"TASK=calc;PID=%d;TIME=%.6f;RESULT=sum_primes_up_to_%d=%ld\n",
getpid(), et, limit, sum);

        write(pipes[i][1], outbuf, strlen(outbuf));

        close(pipes[i][1]);

        exit(0);

    }
} else {

    // PARENT: store child pid, close write end (we'll read)

    pids[i] = pid;

    close(pipes[i][1]);

}

}

// Parent collects results
for (int i = 0; i < NCH; ++i) {

    // wait for each child and get status

    int status;

    pid_t w = waitpid(pids[i], &status, 0);

    if (w == -1) {

        perror("waitpid");

        continue;

    }

    // read data from pipe

    char buf[BUF_SIZE];

    ssize_t n = read(pipes[i][0], buf, sizeof(buf)-1);

    if (n < 0) {

        fprintf(stderr, "Parent: read error from child %d: %s\n", pids[i],
strerror(errno));

```



```

        buf[0] = '\0';
    } else {
        buf[n] = '\0';
    }
    close(pipes[i][0]);

    // Print summary: PID, execution time (in child's message), status
    printf("---- Child report ----\n");
    printf("waitpid returned: %d\n", (int)w);
    if (WIFEXITED(status)) {
        printf("Status: exited normally (code %d)\n", WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("Status: killed by signal %d\n", WTERMSIG(status));
    } else {
        printf("Status: other (status=0x%x)\n", status);
    }
    if (n > 0) {
        printf("Message from child: %s", buf);
    } else {
        printf("Message from child: (no message)\n");
    }
    printf("\n");
}
printf("Parent: all children done. Exiting.\n");
return 0;
}

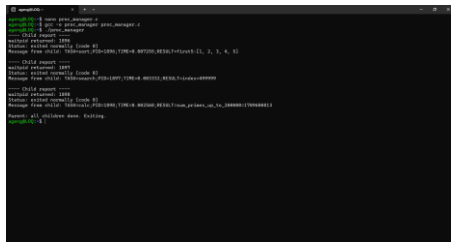
```

2. Compile dan jalankan

```
gcc -o proc_manager proc_manager.c
```

```
./proc_manager
```

3. Dokumentasi



Penjelasan singkat program

- Parent membuat 3 pipe() (satu untuk tiap child) lalu fork() tiga kali.
- Child 0 (sorting): membuat array yang terbalik, menjalankan bubble_sort, lalu menulis hasil ringkasan (5 elemen pertama) dan waktu eksekusi ke pipe.
- Child 1 (searching): membuat array besar, melakukan linear_search target yang berada di akhir (worst-case), menulis indeks hasil dan waktu eksekusi.
- Child 2 (calculation): menghitung jumlah bilangan prima hingga batas tertentu (sieve sederhana), menulis hasil dan waktu eksekusi.
- Masing-masing child mengukur waktu sendiri dengan clock_gettime(CLOCK_MONOTONIC, ...).
- Parent menutup ujung tulis pipe, menunggu (waitpid) tiap child, lalu membaca pesan dari pipe, dan menampilkan:
 - PID (termasuk dalam pesan child),
 - Execution time (termasuk dalam pesan child),

- Status exit (diperoleh dari waitpid + makro WIFEXITED/WEXITSTATUS/WIFSIGNALED).

Tugas 2: Chat Application dengan IPC Implementasikan aplikasi chat sederhana menggunakan: Shared Memory atau Message Queue untuk menyimpan pesan Multiple processes sebagai pengguna Mutex/Semaphore untuk sinkronisasi Fitur: kirim pesan, lihat history, exit

Lahkah Kerja:

1. Buat File chat_ipc.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>    // O_* constants
#include <sys/mman.h>
#include <sys/stat.h> // mode constants
#include <semaphore.h>
#include <time.h>
#include <errno.h>
#include <sys/types.h>
#define SHM_NAME "/chat_shm_example_rafi"
#define SEM_NAME "/chat_sem_example_rafi"
#define MAX_MESSAGES 200
#define MAX_TEXT_LEN 512
#define MAX_NAME_LEN 64
```

```
typedef struct {
    pid_t pid;
    char user[MAX_NAME_LEN];
    time_t ts;
    char text[MAX_TEXT_LEN];
    int valid; // 0 = empty, 1 = used
```

```
} chat_msg_t;
```

```
typedef struct {
```

```
    int head; // index of oldest message
```

```
    int tail; // index after newest message
```

```
    int count; // number of messages currently stored
```

```
    chat_msg_t msgs[MAX_MESSAGES];
```

```
} chat_shm_t;
```

```
/* helper timestamp -> string */
```

```
static void ts_to_str(time_t t, char *buf, size_t n) {
```

```
    struct tm tm;
```

```
    localtime_r(&t, &tm);
```

```
    strftime(buf, n, "%Y-%m-%d %H:%M:%S", &tm);
```

```
}
```

```
/* open or create shared memory and semaphore */
```

```
static int open_resources(chat_shm_t **out_shm, sem_t **out_sem, int  
*created_shm) {
```

```
    int shm_fd;
```

```
    int first_create = 0;
```

```
    shm_fd = shm_open(SHM_NAME, O_RDWR | O_CREAT | O_EXCL, 0666);
```

```
    if (shm_fd >= 0) {
```

```
        first_create = 1;
```

```
        if (ftruncate(shm_fd, sizeof(chat_shm_t)) == -1) {
```

```
            perror("ftruncate");
```

```
            close(shm_fd);
```

```
            shm_unlink(SHM_NAME);
```

```
            return -1;
```

```

    }
} else if (errno == EEXIST) {
    shm_fd = shm_open(SHM_NAME, O_RDWR, 0);
    if (shm_fd < 0) {
        perror("shm_open existing");
        return -1;
    }
} else {
    perror("shm_open");
    return -1;
}

void *map = mmap(NULL, sizeof(chat_shm_t), PROT_READ |
PROT_WRITE, MAP_SHARED, shm_fd, 0);
if (map == MAP_FAILED) {
    perror("mmap");
    close(shm_fd);
    return -1;
}
close(shm_fd);

chat_shm_t *shm = (chat_shm_t *)map;

if (first_create) {
    shm->head = 0;
    shm->tail = 0;
    shm->count = 0;
    for (int i = 0; i < MAX_MESSAGES; ++i) shm->msgs[i].valid = 0;
}

```

```

sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0666, 1);
if (sem == SEM_FAILED) {
    perror("sem_open");
    munmap(map, sizeof(chat_shm_t));
    return -1;
}

*out_shm = shm;
*out_sem = sem;
if (created_shm) *created_shm = first_create;
return 0;
}

/* write message (protected by semaphore) */
static void write_message(chat_shm_t *shm, sem_t *sem, const char *user, const
char *text) {
    sem_wait(sem);
    int pos = shm->tail;
    chat_msg_t *m = &shm->msgs[pos];
    m->pid = getpid();
    strncpy(m->user, user, MAX_NAME_LEN-1);
    m->user[MAX_NAME_LEN-1] = '\0';
    m->ts = time(NULL);
    strncpy(m->text, text, MAX_TEXT_LEN-1);
    m->text[MAX_TEXT_LEN-1] = '\0';
    m->valid = 1;

    shm->tail = (shm->tail + 1) % MAX_MESSAGES;
    if (shm->count < MAX_MESSAGES) {
        shm->count++;
    }
}

```

```

    } else {
        shm->head = (shm->head + 1) % MAX_MESSAGES; // overwrite oldest
    }
    sem_post(sem);
}

```

/* print history in the requested format */

```

static void print_history(chat_shm_t *shm, sem_t *sem) {
    sem_wait(sem);
    printf("=== CHAT ROOM ===\n");
    int idx = shm->head;
    for (int i = 0; i < shm->count; ++i) {
        chat_msg_t *m = &shm->msgs[idx];
        if (m->valid) {
            printf("%s> %s\n", m->user, m->text); // format: User1> pesan
        }
        idx = (idx + 1) % MAX_MESSAGES;
    }
    if (shm->count == 0) {
        printf("(no messages)\n");
    }
    printf("=== end ===\n");
    sem_post(sem);
}

```

/* cleanup resources */

```

static void cleanup_resources() {
    if (shm_unlink(SHM_NAME) == -1) {
        if (errno != ENOENT) perror("shm_unlink");
    }
}

```



```

    } else {
        printf("Shared memory removed.\n");
    }
    if(sem_unlink(SEM_NAME) == -1) {
        if(errno != ENOENT) perror("sem_unlink");
    } else {
        printf("Semaphore removed.\n");
    }
}

int main(int argc, char *argv[]) {
    if(argc == 2 && strcmp(argv[1], "--cleanup") == 0) {
        cleanup_resources();
        return 0;
    }

    chat_shm_t *shm;
    sem_t *sem;
    if(open_resources(&shm, &sem, NULL) != 0) {
        fprintf(stderr, "Gagal membuka resource shared memory / semaphore\n");
        return 1;
    }

    char name[MAX_NAME_LEN];
    printf("Nama Anda: ");
    if(!fgets(name, sizeof(name), stdin)) {
        printf("Input error\n");
        return 1;
    }
}

```

```

name[strcspn(name, "\n")] = '\0';
if (strlen(name) == 0) strncpy(name, "anon", MAX_NAME_LEN);

printf("Halo %s! Ketik 'help' untuk perintah (send/history/exit).\n", name);

char line[1024];
while (1) {
    printf("%s> ", name);
    if (!fgets(line, sizeof(line), stdin)) {
        printf("\nEOF. Keluar.\n");
        break;
    }
    line[strcspn(line, "\n")] = '\0';

    if (strcmp(line, "help") == 0) {
        printf("Perintah:\n");
        printf("  send <pesan> - kirim pesan\n");
        printf("  history      - lihat history chat\n");
        printf("  exit         - keluar\n");
        continue;
    } else if (strncmp(line, "send ", 5) == 0) {
        const char *msg = line + 5;
        if (strlen(msg) == 0) {
            printf("Pesan kosong.\n");
            continue;
        }
        write_message(shm, sem, name, msg);
        continue;
    } else if (strcmp(line, "history") == 0) {

```

```

        print_history(shm, sem);
        continue;
    } else if (strcmp(line, "exit") == 0) {
        printf("Keluar. Bye %s\n", name);
        break;
    } else if (strlen(line) == 0) {
        continue;
    } else {
        printf("Perintah tidak dikenal. Ketik 'help'.\n");
        continue;
    }
}

munmap(shm, sizeof(chat_shm_t));
sem_close(sem);

return 0;
}

```

2. Compile dan jalankan

`gcc -o chat_ipc chat_ipc.c -lrt -pthread`

`./chat_ipc`

3. Dokumentasi

```

smp@ubuntu:~$ gcc -o chat_ipc chat_ipc.c -lrt -pthread
smp@ubuntu:~$ ./chat_ipc
Nama Anda: ageng
Halo ageng! Ketik 'help' untuk perintah (send/history/exit).
ageng> help
Perintah:
  send <pesan> - kirim pesan
  history      - lihat history chat
  exit        - keluar
ageng> send
Perintah tidak dikenal. Ketik 'help'.
ageng> help
Perintah:
  send <pesan> - kirim pesan
  history      - lihat history chat
  exit        - keluar
ageng> pesan
Perintah tidak dikenal. Ketik 'help'.
ageng> send <halo juga bayu>
ageng> send <hujan disini>
ageng> history
=== CHAT ROOM ===
bayu> <halo gengg>
ageng> <halo juga bayu>
bayu> <ditara hujan>
ageng> <hujan disini>
=== end ===
bayu> exit
Keluar. Bye bayu
smp@ubuntu:~$

```

Penjelasan singkat program

1. Inisialisasi Resource

- Shared memory (/chat_shm_example_rafi) dibuat menggunakan shm_open().
- Semaphore (/chat_sem_example_rafi) dibuat dengan sem_open().
- Jika sudah ada, proses lain akan menggunakan resource yang sama.

2. Struktur Data

- chat_msg_t → menyimpan satu pesan (nama user, teks, PID, timestamp).
- chat_shm_t → buffer melingkar (circular buffer) yang menyimpan semua pesan (msgs[]).

3. Operasi Utama

- send <pesan>
→ Menulis pesan ke shared memory, dilindungi dengan sem_wait() dan sem_post() agar aman dari race condition.
- history
→ Menampilkan seluruh pesan dari shared memory.
- exit
→ Keluar dari aplikasi.
- --cleanup
→ Menghapus shared memory dan semaphore dari sistem.

4. Sinkronisasi

- Saat proses menulis atau membaca pesan, semaphore dikunci (sem_wait).
- Setelah selesai, dikembalikan (sem_post), agar proses lain bisa mengakses.

